CS25C09 – JAVA PROGRAMMING

25 Important 3-Mark Questions with Answers
Unit I: Foundations of OOP and Java

1. What is Object-Oriented Programming (OOP)? List its principles.

Object-Oriented Programming (OOP) is a programming approach that organizes a program using classes and objects.
The four main principles are:
1. Encapsulation – Combining data and methods into a single unit.
2. Abstraction – Hiding implementation details.
3. Inheritance – Acquiring properties and methods from another class.
4. Polymorphism – One interface or method name can represent different behaviours.

2. What is JDK? Differentiate JDK, JRE and JVM.

• JDK (Java Development Kit): Provides tools to develop Java programs, including the compiler.
• JRE (Java Runtime Environment): Provides the environment and libraries needed to run Java applications.
• JVM (Java Virtual Machine): Executes Java bytecode.
JDK includes development tools and runtime components; conceptually, the JRE contains the JVM and supporting libraries.

3. What is bytecode? Why is Java called platform-independent?

1. Bytecode is the intermediate code generated by the Java compiler (javac).
2. It is stored in a .class file.
3. The JVM on a compatible platform executes the bytecode, supporting Java's principle of Write Once, Run Anywhere (WORA).

4. What are primitive data types in Java? List them.

Primitive data types are built-in types that store simple values. Java has eight primitive types:
• Integer: byte, short, int, long
• Floating-point: float, double
• Character: char
• Boolean: boolean
String is not a primitive type; it is a class.

5. Define a variable. List the types of variables in Java.

A variable is a named storage location used to hold a value.
1. Local variable: Declared inside a method or block.
2. Instance variable: Belongs to an object; each object has its own instance fields.
3. Static variable: Belongs to the class and is shared by its instances.
Example: int mark = 90; (instance field if declared in a class), static String college = "AU"; (static field), and int age = 20; inside a method (local variable).

6. What is an array? How is it declared and initialized in Java?

An array is an object that stores a fixed number of elements of the same type.

```
int marks[] = {80, 90, 75};
System.out.println(marks[0]);
```

Output: `80`
Array indexing starts from 0. The array length is obtained using marks.length.

7. List the different types of operators in Java.

Major categories include:
1. Arithmetic: +, -, *, /, %
2. Relational: >, <, ==, !=
3. Logical: &&, ||, !
4. Assignment: =, +=, -=
5. Unary: ++, --
6. Bitwise: &, |, ^
7. Conditional (ternary): ?:

8. What are control statements? List their types.

Control statements determine the order in which program statements execute.
1. Decision-making: if, if-else, switch
2. Looping: for, while, do-while
3. Jump statements: break, continue, return
Example: if (mark >= 50) executes its block when the condition is true.

9. Define a class and an object with an example.

• Class: A blueprint that defines data and methods.
• Object: An instance of a class.

```
class Student {
  int mark = 90;
}
Student s = new Student();
System.out.println(s.mark);
```

Output: `90`. Student is the class and s refers to the created object.

10. What is a constructor? State its characteristics.

A constructor is a special member used to initialize an object when it is created.
1. Its name must be the same as the class name.
2. It has no return type, not even void.
3. It is invoked when an object is created using new.
Example: `Student() { System.out.println("Object created"); }`

11. Differentiate between a constructor and a method.

• A constructor has the same name as the class; a method can have any valid name.
• A constructor has no return type; a method may have a return type or void.
• A constructor is invoked during object creation; a method is called to perform an operation.

12. What is a default constructor? Give an example.

A default constructor is the no-argument constructor automatically supplied by the compiler when a class declares no constructor.
Example:

```
class Student { int mark; }
Student s = new Student();
System.out.println(s.mark);
```

Output: `0`. The instance variable mark receives its default value, 0. If a constructor is explicitly declared, the compiler does not automatically supply the default constructor.

13. What is a parameterized constructor? Give an example.

A parameterized constructor accepts arguments to initialize an object's data.

```
class Student {
  int mark;
  Student(int m) { mark = m; }
}
```

Creating an object with `new Student(90)` initializes mark to 90.

14. What is constructor overloading?

Constructor overloading means defining multiple constructors in the same class with different parameter lists.

```
class Student {
  Student() { System.out.println("No arguments"); }
  Student(int mark) { System.out.println(mark); }
}
```

The appropriate constructor is selected based on the arguments supplied during object creation.

15. What is a static variable? State its advantages.

A static variable is a class-level variable shared by all instances of that class.
Example: `static String college = "AU";`
Advantages:
1. A single shared copy is maintained for the class.
2. It avoids a separate copy in every object.
3. It can be accessed using the class name, for example Student.college.

16. What is a static method? State its limitations.

A static method belongs to the class rather than to an individual object.
Example: `static void display() { System.out.println("Java"); }`
It can be called using Demo.display().
Limitations:
1. It cannot directly access instance variables or instance methods.
2. It cannot use this or super directly.
3. It can directly access static members of the class.

17. What is a static block in Java?

A static block initializes static data. It executes when the class is initialized, before its static members are used for the first time as part of that initialization.
Example:

```
class Demo {
  static int value;
  static { value = 100; }
}
```

Here, value is initialized to 100 by the static block.

18. What is method overriding? Give an example.

Method overriding occurs when a subclass provides its own implementation of an inherited instance method with a compatible method signature.

```
class Animal {
  void sound() { System.out.println("Animal sound"); }
}
class Dog extends Animal {
  @Override
  void sound() { System.out.println("Bark"); }
}
```

Calling sound() on a Dog object prints Bark. Overriding supports runtime polymorphism.

19. Differentiate between method overloading and method overriding.

- Overloading: Same method name with different parameter lists, usually within the same class.
- Overriding: A subclass redefines an inherited instance method with a compatible signature.
- Overloading is resolved using method arguments at compile time; overriding enables runtime polymorphism.

20. What are access specifiers in Java? List them.

Access specifiers control the visibility of classes, variables, constructors and methods.

1. private – Accessible within the declaring class.
2. Default (no keyword) – Accessible within the same package.
3. protected – Accessible within the same package and through inheritance subject to Java's access rules.
4. public – Accessible from anywhere the declaring type and member are accessible.

21. What is inheritance? State its advantages.

Inheritance is an OOP feature in which a subclass acquires accessible members and behaviour from a superclass.
Example: `class Dog extends Animal { }`
Advantages:

1. Promotes code reusability.
2. Reduces code duplication.
3. Supports method overriding and runtime polymorphism.

22. List the types of inheritance supported in Java.

The main types of inheritance are:

1. Single inheritance: One subclass inherits from one superclass.
2. Multilevel inheritance: A class inherits through a chain of classes.
3. Hierarchical inheritance: Multiple subclasses inherit from one superclass.

Java does not support multiple inheritance of classes, but a class can implement multiple interfaces.

23. What is the purpose of the extends keyword?

The extends keyword establishes inheritance between a subclass and a superclass.
Example:

```
class Animal {
  void eat() { System.out.println("Eating"); }
}
class Dog extends Animal { }
```

Dog inherits the accessible eat() method from Animal and can use it without redefining it.

24. What is the purpose of the super keyword?

The super keyword refers to the immediate superclass. Its main uses are:

1. Accessing a superclass variable hidden by a subclass variable.
2. Calling an overridden superclass method.
3. Calling a superclass constructor using super() or super(arguments).

Example: In a subclass constructor, super(); calls the no-argument superclass constructor. It must be the first statement in a constructor when explicitly used.

25. Write a Java program to demonstrate inheritance and constructor execution.

Program:

```
class Person {
  Person() { System.out.println("Person constructor"); }
}
class Student extends Person {
  Student() { System.out.println("Student constructor"); }
}
public class Main {
  public static void main(String[] args) {
    Student s = new Student();
  }
}
```

Output:

```
Person constructor
Student constructor
```

Explanation: Student inherits from Person. When a Student object is created, the superclass constructor executes first, followed by the subclass constructor.